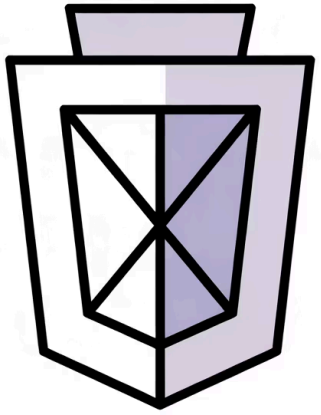
JavaScript - Jour 8

Le design pattern MVC côté client

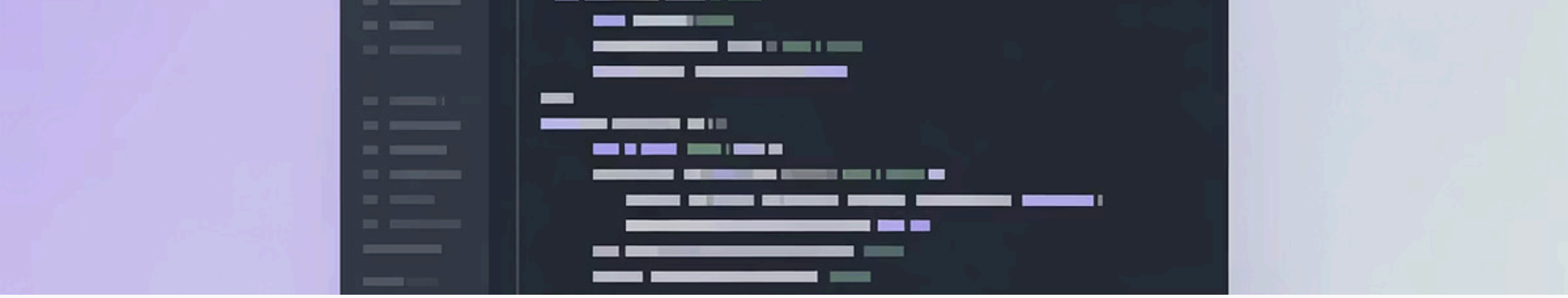


Objectifs pédagogiques de la journée

- Introduction au pattern MVC
- Le modèle (model) - Gestion des données
- Le manager - Gestion de collections et logique métier avancée
- Le pattern Observer : EventEmitter
- La vue (view) - Affichage et génération du HTML
- Le contrôleur (controller) - Coordination et orchestration

Avant de continuer…

On va s’installer initialiser un projet avec un outil très utile et utilisé !



Vite, le bundler JS front-end puissant et rapide

Vite est un bundler JavaScript conçu par le même créateur que le framework VueJS.
Il permet de gérer des projets front-end complexes avec une configuration simpliste.

Étapes

1

Ouvrir un terminal (Powershell ou Bash/ZSH)

Placez-vous ensuite dans le répertoire dans lequel vous voulez créer le nouveau projet.

2

Exécutez la commande suivante dans le terminal

`npm create vite`

Remplissez ensuite le questionnaire dans le terminal avec les paramètres souhaités.

3

Ouvrez votre éditeur de code dans le dossier du projet

`code vite-project` (VS Code) ou `webstorm vite-project` (WebStorm)

4

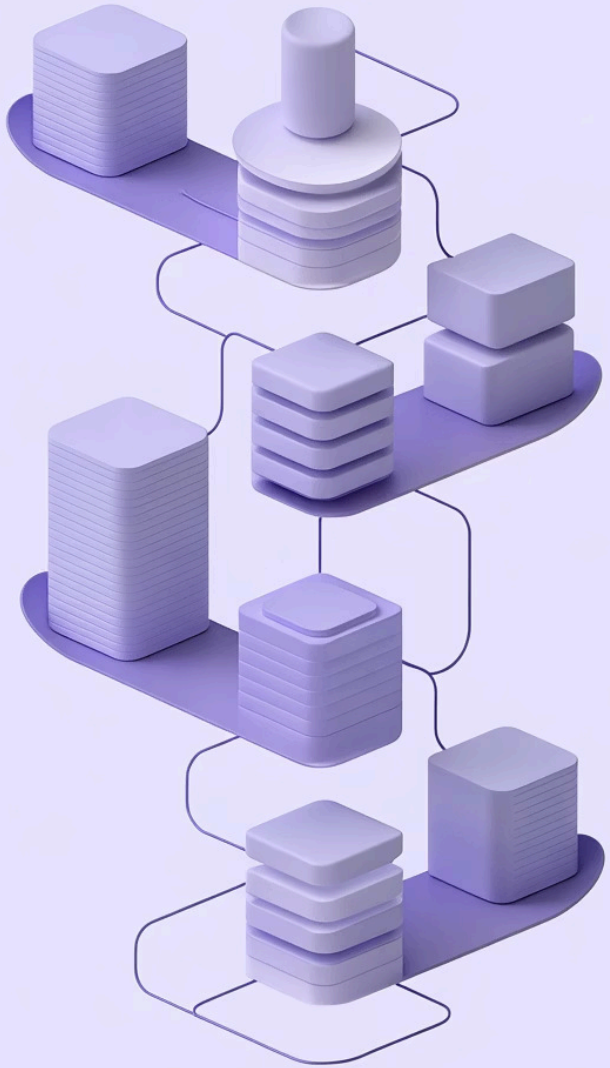
Installez les dépendances du projet

À la racine de votre projet (obligatoire), lancez la commande `npm install` pour installer toutes les dépendances nécessaires pour le projet.

5

Exécutez le projet

`npm run dev` lancera le projet et ouvrira un serveur sur votre machine pour pouvoir y accéder dans un navigateur.



MODULE

Introduction au pattern MVC

Le défi du code monolithique

Imaginez écrire tout votre code JavaScript dans un seul fichier : la logique métier, l'affichage, la gestion des événements, tout mélangé. Au début, avec 50 lignes, ça fonctionne. Mais à 500 lignes ? À 5000 lignes ?

Code ingérable

Difficile de localiser les fonctionnalités ou de comprendre les interactions.

Dépendances fragiles

Modifier une partie risque de casser d'autres fonctionnalités inattendues.

Collaboration complexe

Travailler en équipe sur le même fichier gigantesque devient un cauchemar.

Tests impossibles

Le code est trop entrelacé pour être testé efficacement et isolément.

Développement lent

Ajouter de nouvelles fonctionnalités prend de plus en plus de temps et d'efforts.

Le pattern MVC (Model-View-Controller) résout ces problèmes en séparant clairement les responsabilités, rendant le code plus **organisé**, **maintenable** et **évolutif**.

Le pattern MVC : Modèle-Vue-Contrôleur

Le MVC (Model-View-Controller) est un pattern d'architecture logiciel qui organise le code en trois couches distinctes, chacune ayant une responsabilité précise et bien définie.

Le modèle (Model)

Gère les données de l'application, la logique métier et les règles de validation. Il est indépendant de l'interface utilisateur.

La vue (View)

Responsable de l'affichage des données provenant du Modèle et de la présentation de l'interface utilisateur à l'utilisateur.

Le contrôleur (Controller)

Agit comme un intermédiaire, recevant les entrées de l'utilisateur, les traitant et mettant à jour le modèle ou la vue en conséquence.

Principe fondamental : Chaque couche ne connaît que ce qu'elle doit connaître et fait uniquement ce qu'elle doit faire.

Analogie : Le restaurant

Pour mieux comprendre les rôles du modèle, de la vue et du contrôleur, imaginons le fonctionnement d'un restaurant bien organisé.

Le modèle (cuisine)

Les cuisiniers préparent les plats selon des recettes précises. Ils gèrent les ingrédients (données) et appliquent les techniques de cuisine (logique métier), sans se soucier de qui mange ou de la présentation.

La vue (salle)

Le serveur présente les plats aux clients de manière élégante. Il ne sait pas comment les plats ont été préparés et ne décide pas du menu. Son rôle est uniquement l'aspect visuel et la présentation.

Le contrôleur (maître d'hôtel)

Il prend la commande du client, la transmet à la cuisine, puis demande au serveur de présenter le plat une fois prêt. Il coordonne tout sans cuisiner ni servir lui-même, assurant la fluidité du service.

MVC en action : l'application To-Do

Appliquons le pattern MVC à notre projet de gestion de tâches pour clarifier les responsabilités de chaque composant.

Le modèle

Gère les données des tâches (ajout, suppression, modification) et la logique métier associée.

La vue

Affiche les tâches dans le HTML et génère l'interface utilisateur visible.

Le contrôleur

Interprète les actions de l'utilisateur, demande au modèle de modifier les données, puis actualise la Vue.

Les avantages du pattern MVC

- **Séparation des responsabilités**

  Chaque composant (modèle, vue, contrôleur) a un rôle unique et clairement défini, ce qui rend le code plus modulaire et facile à comprendre.

- **Testabilité**

  Les composants peuvent être testés de manière isolée, garantissant une meilleure fiabilité de l'application et une détection précoce des problèmes.

- **Travail en équipe simplifié**

  Plusieurs développeurs peuvent travailler simultanément sur différentes couches sans se gêner, améliorant la productivité collective.

- **Maintenabilité accrue**

  La structure claire facilite la localisation des bugs et leur correction, réduisant ainsi le temps de maintenance.

- **Réutilisabilité**

  Un même modèle peut être utilisé avec plusieurs vues différentes, permettant de créer diverses interfaces pour les mêmes données.

- **Évolutivité**

  Ajouter de nouvelles fonctionnalités ou modifier des existantes devient plus simple et moins risqué grâce à la modularité du code.

MVC en action : Exemple de modèle simple

```
// Modèle : Gère les données
class DataModel {
  constructor() {
    this.data = [];
  }

  add(item) {
    this.data.push(item);
  }

  getAll() {
    return this.data;
  }
}
```

MVC en action : Exemple de vue simple

```
// Vue : Gère l'affichage
class DisplayView {
  constructor(containerSelector) {
    this.container = document.querySelector(containerSelector);
  }

  render(data) {
    this.container.innerHTML = "";
    data.forEach(item => {
      const element = document.createElement("div");
      element.textContent = item;
      this.container.appendChild(element);
    });
  }
}
```

MVC en action : Exemple de contrôleur simple

```
// Contrôleur : Coordonne le modèle et la vue
class AppController {
  constructor(model, view) {
    this.model = model;
    this.view = view;
  }

  addItem(item) {
    this.model.add(item);
    this.view.render(this.model.getAll());
  }
}
```

Analyser une application monolithique

Objectif : Identifier les différentes responsabilités dans du code non organisé.

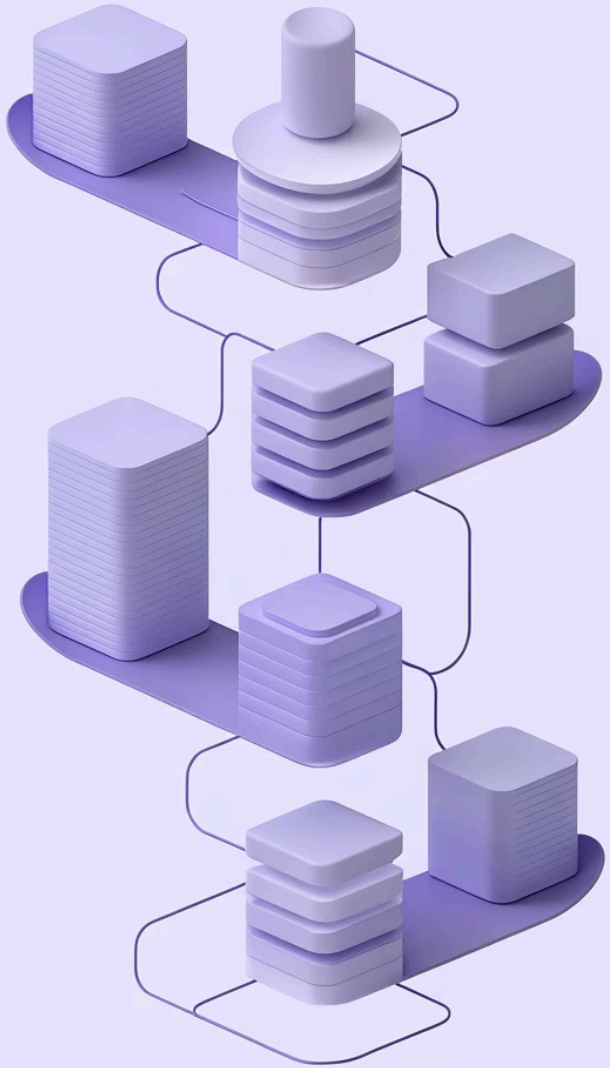
Voici une petite application de gestion de notes écrite de manière monolithique (tout dans un seul bloc) :

▼ Code monolithique (code "spaghetti")

```
const notes = [];

document.querySelector("#add-note").addEventListener("click", () => {
  const title = document.querySelector("#note-title").value;
  const content = document.querySelector("#note-content").value;

  if (title.trim() === "") {
    alert("Le titre ne peut pas être vide");
    return;
  }

  const note = {
    id: Date.now(),
    title: title.trim(),
    content: content.trim(),
    createdAt: new Date()
  };

  notes.push(note);

  const notesList = document.querySelector("#notes-list");
  notesList.innerHTML = "";

  notes.forEach(note => {
    const noteElement = document.createElement("div");
    noteElement.classList.add("note");
    noteElement.innerHTML = `
      <h3>${note.title}</h3>
      <p>${note.content}</p>
      <button onclick="deleteNote(${note.id})">Supprimer</button>
    `;
    notesList.appendChild(noteElement);
  });

  document.querySelector("#note-title").value = "";
  document.querySelector("#note-content").value = "";

  localStorage.setItem("notes", JSON.stringify(notes));
});

function deleteNote(id) {
  const index = notes.findIndex(n => n.id === id);
  notes.splice(index, 1);

  const notesList = document.querySelector("#notes-list");
  notesList.innerHTML = "";

  notes.forEach(note => {
    const noteElement = document.createElement("div");
    noteElement.classList.add("note");
    noteElement.innerHTML = `
      <h3>${note.title}</h3>
      <p>${note.content}</p>
      <button onclick="deleteNote(${note.id})">Supprimer</button>
    `;
    notesList.appendChild(noteElement);
  });

  localStorage.setItem("notes", JSON.stringify(notes));
}
```

Tâche : Identifiez dans ce code

- Les parties qui gèrent les données (**Model**)
- Les parties qui gèrent l'affichage (**View**)
- Les parties qui coordonnent les interactions (**Controller**)
- Le code dupliqué qui pourrait être factorisé
- Les problèmes de cette approche monolithique



MODULE

Le modèle (model) - Gestion des données

Le modèle (Model) : Rôle et principes

Le modèle est responsable de la persistance des données, de leur validation et de toute la logique métier complexe. Il est le cerveau de l'application, indépendant de la manière dont les informations sont présentées ou des interactions utilisateur.

✓ CONTIENT

Les données de l'application et leur structure.

La logique métier (règles, validations, calculs complexes).

La validation des données pour assurer leur intégrité.

La capacité d'émettre des événements pour notifier les changements.

✝ NE CONTIENT PAS

La manipulation directe du DOM.

La connaissance de l'interface utilisateur ou de son état.

Une dépendance envers la vue (elle ne sait pas comment ses données sont affichées).

Analogie : Le modèle est un coffre-fort

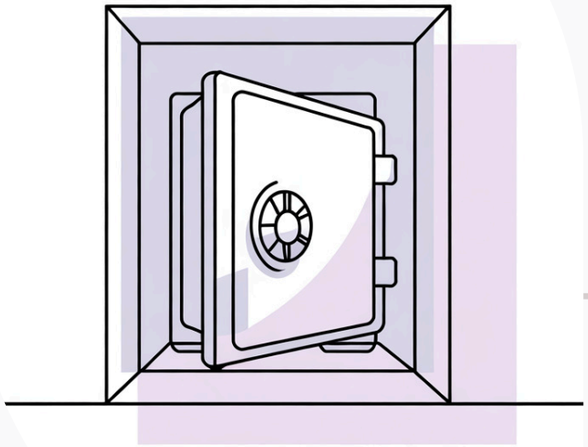
Comme le coffre-fort, le modèle est le gardien de l'information, qui garanti son intégrité et sa sécurité, sans se soucier de la manière dont elle est présentée ou utilisée par l'interface utilisateur.

Contient des objets

Les données essentielles

Règles d'accès

Validation stricte



Historique

Transactions et événements

Indépendant

Séparé de l'affichage

EXEMPLE

Le modèle : Exemple simple

Voici un exemple concret d'un modèle représenté par la classe `Product`. Il encapsule les données du produit et les règles métiers associées, sans aucune connaissance de l'interface utilisateur.

▼ Classe Product (modèle)

```
class Product {
  #id;
  #name;
  #price;
  #stock;

  constructor(name, price, stock = 0) {
    // Validation des données
    if (price < 0) {
      throw new Error("Le prix ne peut pas être négatif");
    }

    this.#id = crypto.randomUUID();
    this.#name = name;
    this.#price = price;
    this.#stock = stock;
  }

  get id() {
    return this.#id;
  }

  get name() {
    return this.#name;
  }

  get price() {
    return this.#price;
  }

  get stock() {
    return this.#stock;
  }

  // Logique métier : Acheter un produit
  purchase(quantity) {
    if (quantity > this.#stock) {
      throw new Error("Stock insuffisant");
    }
    this.#stock -= quantity;
  }

  // Logique métier : Ajouter du stock
  restock(quantity) {
    if (quantity < 0) {
      throw new Error("La quantité doit être positive");
    }
    this.#stock += quantity;
  }

  // Logique métier : Calculer le prix total
  getTotalValue() {
    return this.#price * this.#stock;
  }

  toJSON() {
    return {
      id: this.#id,
      name: this.#name,
      price: this.#price,
      stock: this.#stock,
    };
  }
}
```

Gestion des Données

Les attributs `id`, `name`, `price` et `stock` sont gérés directement par le modèle.

Validation Intégrée

Les contrôles comme `price < 0` ou `quantity > this.#stock` garantissent la cohérence des données.

Logique Métier

Les méthodes `purchase()`, `restock()` et `getTotalValue()` implémentent le comportement du produit.

MISE EN PRATIQUE

Créer le modèle Task

Objectif : Créer une classe `Task` qui représente une tâche dans notre application To-Do, en respectant les principes du pattern Model.

Voici le cahier des charges détaillé pour la classe `Task` :

PROPRIÉTÉS REQUISES

- Un identifiant unique (`id`)
- Un titre (`title`)
- Un état de complétion (`completed`, un booléen)
- Une date de création (`createdAt`)

FONCTIONNALITÉS ET MÉTHODES

- Validation : Le titre ne doit pas être vide.
- Encapsulation : Utiliser des propriétés privées.
- Accesseurs : Fournir des getters pour toutes les propriétés.
- Méthode `toggle()` : Pour basculer l'état de complétion.
- Méthode `updateTitle(newTitle)` : Pour modifier le titre (avec validation).
- Sérialisation : `toJSON()` pour convertir l'objet en JSON.
- Désérialisation : Méthode statique `fromJSON(json)`.

Étapes de développement du modèle Task

Initialisation du fichier

Créer le fichier `src/models/Task.js` pour la définition de notre modèle.

Propriétés privées

Définir les propriétés internes (`id`, `title`, `completed`, `createdAt`) comme étant privées.

Constructeur & validation

Écrire le constructeur pour initialiser la tâche et valider que le titre n'est pas vide.

Accesseurs (getters)

Fournir des méthodes `get` pour accéder aux valeurs des propriétés de manière contrôlée.

Logique métier

Implémenter `toggle()` pour inverser l'état de réalisation de la tâche.

Sérialisation

Ajouter `toJSON()` et la méthode statique `fromJSON()`.

Test local

Tester le fonctionnement de la classe `Task` directement dans la console de votre navigateur.

EXERCICE

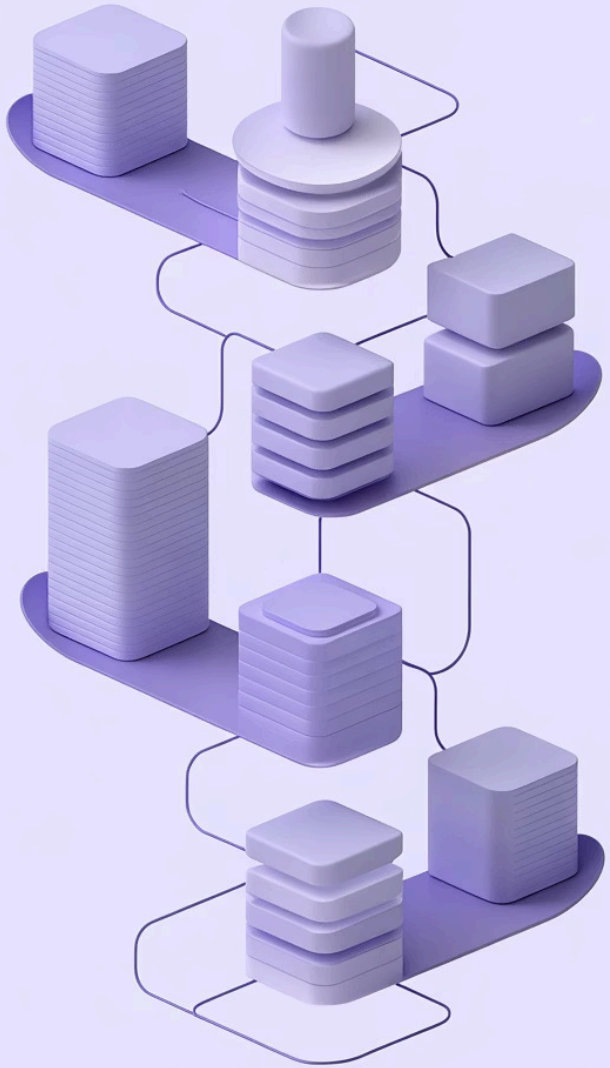
Mise à jour du titre de la tâche

Pour enrichir notre modèle `Task` et de simuler une logique métier plus complexe, nous allons lui ajouter une nouvelle fonctionnalité.

Votre défi : Implémenter une méthode `updateTitle(newTitle)` dans la classe `Task`. Cette méthode devra permettre de modifier le titre d'une tâche, tout en garantissant la validité de cette modification.

Voici les exigences détaillées pour cette méthode :

- **Validation du titre :** Avant toute affectation, le `newTitle` doit être "trimé" (les espaces au début et à la fin doivent être supprimés). Si après cette opération le titre est une chaîne vide, la méthode doit lever une erreur (par exemple, `throw new Error("Le titre ne peut pas être vide.")`).
- **Encapsulation :** Assurez-vous que la modification respecte le principe d'encapsulation en agissant sur la propriété privée `#title`.
- **Test :** Après implémentation, testez cette nouvelle méthode directement dans la console de votre navigateur pour vérifier son bon fonctionnement et la robustesse de sa validation.



MODULE

Le manager - Gestion de collections et logique métier avancée

Le manager : Orchestrateur du modèle

Le manager, parfois appelé Repository ou Service, est une extension de la couche modèle. Il centralise la gestion des collections d'objets (modèles) et orchestre la logique métier avancée.

Gère une collection

Maintient une collection organisée d'objets, facilitant leur accès et leur manipulation.

Opérations CRUD

Centralise toutes les opérations de création, lecture, mise à jour et suppression des objets.

Logique métier avancée

Implémente des fonctionnalités complexes comme les filtres, les statistiques, la recherche et les agrégations de données.

Émet des événements

Notifie les autres parties de l'application des changements survenus au sein de la collection ou des objets.

Pourquoi un manager en plus du modèle ?

Modèle (Task)

Représente **une** tâche individuelle.

Manager (TaskManager)

Gère **toutes** les tâches.

Pour pouvoir respecter le principe de responsabilité unique, il est préférable de faire en sorte que chaque classe fasse une chose et surtout : la fait bien.

Analogie : Le bibliothécaire

Si notre `Task` est un livre, alors le `TaskManager` est le bibliothécaire :

Gère la collection

Il organise et maintient l'ensemble des tâches de l'application.

Recherche par ID

Il sait retrouver chaque tâche spécifique grâce à son identifiant unique.

Filtrage et tri

Il peut organiser les tâches selon divers critères (achevées, en cours, urgentes).

Statistiques

Il fournit des informations agrégées, comme le nombre de tâches terminées ou restantes.

Émission d'événements

Il signale quand une tâche est ajoutée, modifiée ou supprimée, permettant aux autres composants de réagir.

Ne modifie pas les tâches

Le manager ne réécrit pas une tâche elle-même. Il délègue les changements internes à l'objet `Task`.

EXEMPLE DE CODE

Implémentation du manager : ProductManager

Voici un exemple concret de classe `ProductManager`, qui gère une collection de produits et met en œuvre les principes d'encapsulation et de logique métier avancée.

▼ Exemple de manager de produits

```
class ProductManager extends EventEmitter {
  #products;

  constructor() {
    super();
    this.#products = [];
  }

  // CREATE
  add(name, price, stock) {
    const product = new Product(name, price, stock);
    this.#products.push(product);
    this.emit("product:added", { product });
    return product;
  }

  // READ
  getById(id) {
    return this.#products.find(p => p.id === id);
  }

  getAll() {
    return this.#products.map(p => p.toJSON());
  }

  // UPDATE
  updatePrice(id, newPrice) {
    const product = this.getById(id);
    if (!product) throw new Error("Produit introuvable");

    product.price = newPrice;
    this.emit("product:updated", { product });
  }

  // DELETE
  remove(id) {
    const index = this.#products.findIndex(p => p.id === id);
    if (index === -1) return false;

    const removed = this.#products.splice(index, 1)[0];
    this.emit("product:removed", { product: removed });
    return true;
  }

  // LOGIQUE MÉTIER : Filtres
  getInStock() {
    return this.#products.filter(p => p.stock > 0);
  }

  getOutOfStock() {
    return this.#products.filter(p => p.stock === 0);
  }

  // LOGIQUE MÉTIER : Statistiques
  getTotalValue() {
    return this.#products.reduce((sum, p) => p.getTotalValue() + sum, 0);
  }

  getStats() {
    return {
      total: this.#products.length,
      inStock: this.getInStock().length,
      outOfStock: this.getOutOfStock().length,
      totalValue: this.getTotalValue(),
    };
  }
}
```

Pour l'instant, ne nous préoccupons pas de l'extends EventEmitter : ce sera à nous de le mettre en place !

MISE EN PRATIQUE

Créer le TaskManager

Voici le cahier des charges détaillé pour l'implémentation du `TaskManager` :

- **Fondation :** Hériter d'`EventEmitter` (que nous créerons plus tard) pour la communication interne.
- **Stockage :** Maintenir un tableau privé (`#tasks`) de toutes les instances de `Task`.
- **Opérations CRUD :**
  - `add(title)` : Crée et ajoute une nouvelle tâche.
  - `remove(taskId)` : Supprime une tâche spécifique.
  - `toggle(taskId)` : Inverse l'état (complétée/active) d'une tâche.
  - `update(taskId, newTitle)` : Modifie le titre d'une tâche.
- **Requêtes :**
  - `getById(taskId)` : Récupère une tâche par son identifiant.
  - `getAll()` : Retourne toutes les tâches.
  - `getCompleted()` : Récupère les tâches marquées comme complétées.
  - `getActive()` : Récupère les tâches non complétées.

Étapes de développement du `TaskManager`

Pour construire notre `TaskManager`, nous allons suivre une approche structurée, étape par étape :

Créer le fichier `src/managers/TaskManager.js`

Mettre en place la structure de base du fichier pour notre classe.

Définir la classe qui hérite d'`EventEmitter`

Établir l'architecture de la classe (méthodes vides pour le moment).

Implémenter les opérations CRUD avec émission d'événements

Développer les méthodes pour ajouter, supprimer, modifier et basculer des tâches, en notifiant les autres composants via des événements.

Implémenter les méthodes de requête

Ajouter des fonctionnalités pour récupérer des tâches spécifiques par ID, toutes les tâches, ou des sous-ensembles (complétées, actives).

MISE EN PRATIQUE

Implémentation du TaskManager : À vous de continuer

Continuez avec les exigences pour notre classe `TaskManager` :

STATISTIQUES

`getStats()` : Fournit un objet récapitulatif (total, complétées, actives, taux de complétion).

ÉVÉNEMENTS

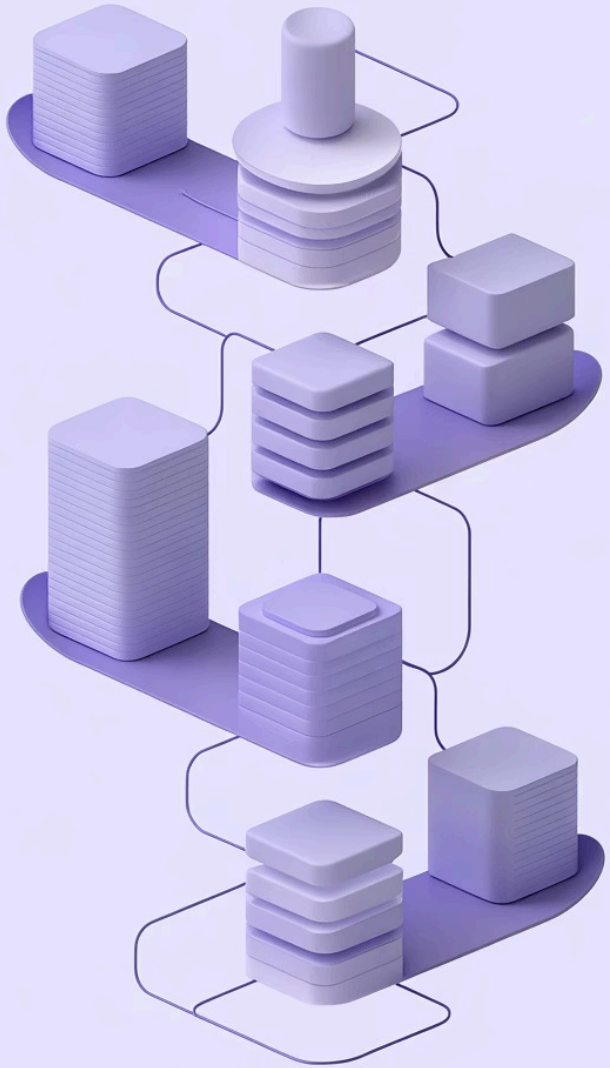
Émettre des événements spécifiques (`"task:added"`, `"task:removed"`, `"task:toggled"`, `"task:updated"`) à chaque modification.

Émettre un événement générique `"tasks:changed"` après toute modification majeure.

PERSISTANCE

`toJSON()` : Sérialise toutes les tâches en format JSON.

`loadFromJSON(jsonTasks)` : Charge et reconstitue les tâches à partir d'un tableau JSON.



MODULE

Le pattern Observer : EventEmitter

Le pattern Observer : explication synthétique

Le pattern Observer permet à des objets (les observateurs) d'être notifiés automatiquement quand un autre objet (le sujet) change d'état. L'avantage clé est que le sujet n'a pas besoin de connaître les détails de ses observateurs, ce qui favorise une architecture logicielle souple et maintenable.

Avantages clés

Découplage

Le manager ne connaît pas directement le contrôleur, ni tout autre observateur, réduisant les dépendances.

Flexibilité

Vous pouvez ajouter ou retirer des observateurs à tout moment, sans modifier le sujet observé.

Réactivité

Les changements d'état du sujet sont propagés automatiquement et instantanément à tous les observateurs.

Analogie : La newsletter

Pour comprendre le pattern Observer, imaginons le fonctionnement d'un journal avec sa newsletter :

- Le journal publie des articles qui sont les **événements**.
- Des lecteurs s'abonnent à la newsletter, agissant comme des **observateurs**.
- Quand un nouvel article est publié, tous les abonnés reçoivent une **notification**.
- Le journal (le sujet) n'a pas besoin de connaître personnellement chaque abonné.
- Les abonnés peuvent se désabonner à tout moment sans affecter le journal.
- De nouveaux abonnés peuvent être ajoutés sans modifier la logique interne du journal.

EXEMPLE

L'EventEmitter en JavaScript

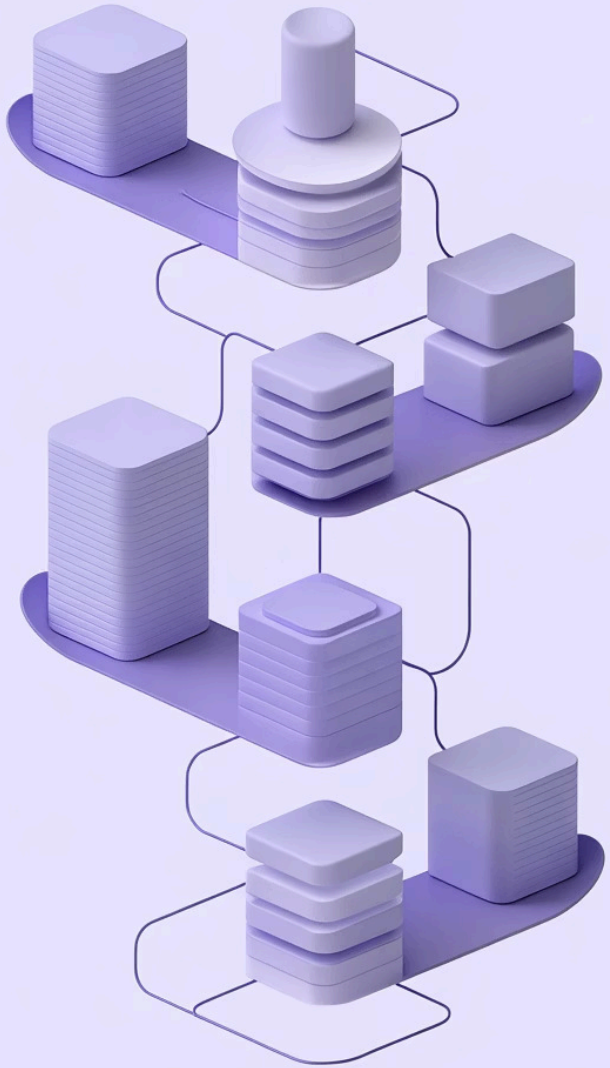
▼ **Code de la classe EventEmitter**

```
class EventEmitter {
  #events;

  constructor() {
    this.#events = new Map();
  }

  // S'abonner à un événement
  on(eventName, callback) {
    if (!this.#events.has(eventName)) {
      this.#events.set(eventName, []);
    }
    this.#events.get(eventName).push(callback);
  }

  // Émettre un événement
  emit(eventName, data) {
    if (!this.#events.has(eventName)) return;

    this.#events.get(eventName).forEach((callback) => {
      callback(data);
    });
  }

  // Se désabonner d'un événement
  off(eventName, callback) {
    if (!this.#events.has(eventName)) return;

      this.#events.set(
        eventName,
        this.#events.get(eventName).filter((cb) => cb !== callback)
      );
    }
  }

// Utilisation
const emitter = new EventEmitter();

// Écouter un événement
emitter.on("message", (data) => {
  console.log(`Message reçu : ${data.text}`);
});

// Émettre un événement
emitter.emit("message", { text: "Bonjour !" });
// -> Affiche "Message reçu : Bonjour !"
```

En détail, voici ce qu'on y retrouve :

- **Une Map privée pour stocker les événements (`string`) et leurs callbacks (`Function[]`)**
- `on(eventName, callback)` **: enregistre un callback pour un événement**
- `emit(eventName, data)` **: appelle tous les callbacks d'un événement**
- `off(eventName, callback)` **: supprime un callback spécifique**
- `once(eventName, callback)` **: enregistre un callback qui ne sera appelé qu'une fois**
- `removeAllListeners(eventName)` **: supprime tous les listeners d'un événement**
- `listenerCount(eventName)` **: retourne le nombre de listeners pour un événement**

ⓘ **Prenez le code, puis collez le dans un fichier `EventEmitter.js` dans un dossier `utils` à la racine de `src`.**



MODULE

La vue (view) - Affichage et génération du HTML

MODULE

La vue (View) : Rôle et règles d'or

La vue est la couche du pattern MVC qui se charge de la présentation à l'utilisateur. C'est elle qui donne vie à nos applications en gérant l'affichage et l'interactivité.

Plus précisément, la vue gère :

- L'affichage des données dans le DOM
- La génération des éléments HTML dynamiquement
- L'écoute des événements DOM (clics, soumission de formulaire)
- La notification du contrôleur via des callbacks quand des interactions utilisateur se produisent

Règles d'or de la vue

Ce qu'elle FAIT

- Manipule le DOM
- Crée et met à jour des éléments HTML
- Écoute les événements du navigateur (clics, saisies)
- Reçoit les données à afficher en paramètres

Ce qu'elle NE FAIT PAS

- Ne modifie jamais les données directement
- Ne contient pas de logique métier complexe
- Ne connaît pas le modèle
- Ne prend pas de décisions concernant les actions utilisateur

ANALOGIE

Analogie : L'écran de cinéma

Pour mieux comprendre le rôle de la **vue**, imaginez-la comme un écran de cinéma :

- Elle **affiche** les images (données) qu'on lui donne.
- Elle **ne crée pas** le contenu du film (la logique métier).
- Elle peut **détecter** quand quelqu'un appuie sur les boutons de la télécommande (événements DOM).
- Elle **informe** le projectionniste (contrôleur) de ces actions.
- Elle **ne décide pas** quel film jouer : elle affiche juste ce qu'on lui demande.

EXEMPLE

Exemple de vue : ProductListView

Voici une implémentation concrète d'une vue en JavaScript, chargée d'afficher une liste de produits et de gérer les interactions utilisateur :

▼ Classe ProductListView (vue)

```
class ProductListView {
  #containerElement;
  #deleteCallback;

  constructor(containerSelector) {
    this.#containerElement = document.querySelector(containerSelector);
  }

  // Méthode principale : affiche les données
  render(products) {
    this.#containerElement.innerHTML = "";

    if (products.length === 0) {
      this.#renderEmptyState();
      return;
    }

    products.forEach(product => {
      const productElement = this.#createProductElement(product);
      this.#containerElement.appendChild(productElement);
    });
  }

  // Crée un élément HTML pour un produit
  #createProductElement(product) {
    const div = document.createElement("div");
    div.classList.add("product");

    const title = document.createElement("h3");
    title.textContent = product.name;

    const price = document.createElement("span");
    price.textContent = `${product.price}€`;

    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Supprimer";
    deleteBtn.addEventListener("click", () => {
      // Appelle le callback pour notifier le Controller
      if (typeof this.#deleteCallback === "function") {
        this.#deleteCallback(product.id);
      }
    });

    div.append(title, price, deleteBtn);
    return div;
  }

  #renderEmptyState() {
    const message = document.createElement("p");
    message.textContent = "Aucun produit disponible";
    this.#containerElement.appendChild(message);
  }

  // Configure le callback pour la suppression
  onDelete(callback) {
    this.#deleteCallback = callback;
  }
}
```

Points clés de cette implémentation

Manipulation du DOM & HTML

La méthode `render()` et `#createProductElement()` sont dédiées à la création et mise à jour des éléments HTML.

Écoute des événements

Les interactions utilisateur, comme le clic sur le bouton "Supprimer", sont gérées localement via `addEventListener`.

Communication via callback

La vue ne prend pas de décision, elle informe un "auditeur" (le contrôleur) via `this.#deleteCallback` que l'utilisateur souhaite supprimer un produit.

Séparation des préoccupations

Elle ne contient aucune logique métier et ne manipule pas directement les données, mais les reçoit pour affichage.

MODULE

Mise en pratique guidée : Créer TaskFormView

Cahier des charges

Votre `TaskFormView` doit implémenter les fonctionnalités suivantes pour gérer efficacement l'ajout de nouvelles tâches :

- Prendre deux sélecteurs en paramètres : le formulaire (`form`) et le champ de saisie (`input`).
- Écouter l'événement `submit` du formulaire pour capturer les soumissions.
- Empêcher le rechargement de la page par défaut (`preventDefault`) lors de la soumission.
- Récupérer le titre de la tâche saisi par l'utilisateur.
- Appeler un callback fourni par le contrôleur avec le titre de la tâche comme argument.
- Vider automatiquement le champ de saisie après l'ajout réussi d'une tâche.

Elle devra également exposer les méthodes suivantes pour permettre au contrôleur d'interagir avec la vue :

onSubmit(callback)

Permet de définir la fonction de rappel qui sera exécutée lors de la soumission du formulaire.

getTitle()

Récupère la valeur actuelle du champ de saisie du titre de la tâche.

clear()

Vide le champ de saisie, le préparant pour une nouvelle entrée.

showError(message)

Affiche un message d'erreur à l'utilisateur, par exemple si la saisie est invalide.

disable() / enable()

Active ou désactive le champ de saisie et le bouton de soumission pour gérer l'état d'interaction.

MISE EN PRATIQUE

Développement de TaskFormView

01

Création du fichier

Commencez par créer le fichier `src/views/TaskFormView.js` pour y loger la logique de votre vue.

02

Structure HTML

Ajoutez le formulaire et le champ de saisie HTML correspondants dans votre fichier `index.html`, en leur donnant des sélecteurs uniques (ID ou classes).

03

Propriétés privées

Définissez les propriétés privées dans la classe pour stocker les références aux éléments DOM (`formElement`, `inputElement`) et le callback de soumission (`submitCallback`).

04

Initialisation du constructeur

Dans le constructeur, récupérez les éléments DOM à partir des sélecteurs fournis et assignez-les aux propriétés privées.

05

Gestion de la soumission

Implémentez une méthode privée pour écouter l'événement `submit` du formulaire, empêcher le comportement par défaut et appeler le `submitCallback`.

06

Méthodes publiques

Développez les méthodes publiques comme `onSubmit()`, `getTitle()`, `clear()`, `showError()`, `disable()` et `enable()` pour permettre l'interaction du contrôleur avec la vue.

MISE EN PRATIQUE

Mise en pratique autonome

Créer la vue d'affichage pour la liste de tâches

Votre prochaine tâche consiste à créer la classe `TaskListView`, qui sera responsable de l'affichage de la liste des tâches dans l'interface utilisateur. Suivez les spécifications détaillées ci-dessous pour son implémentation.

Initialisation et rendu

- Le constructeur prend un sélecteur de conteneur (`containerSelector`).
- La méthode `render(tasks)` vide le conteneur existant, puis affiche un message si la liste de tâches est vide.
- Pour chaque tâche fournie, elle crée un élément `<li>` dynamique.

Structure d'une tâche

Chaque élément `<li>` représentant une tâche doit contenir :

- Une case à cocher (checkbox) pour marquer la tâche comme complétée.
- Le titre de la tâche.
- Un bouton "Éditer".
- Un bouton "Supprimer".

Méthodes de callback

La vue doit exposer les méthodes suivantes pour informer le contrôleur des interactions utilisateur :

- `onToggle(callback)` : pour le changement d'état (complété/non complété).
- `onEdit(callback)` : pour l'édition d'une tâche.
- `onDelete(callback)` : pour la suppression d'une tâche.

Gestion du style

La vue doit appliquer des styles conditionnels pour une meilleure lisibilité :

- Ajouter la classe CSS `task--completed` à l'élément `<li>` si la tâche est complétée.

Idée d'implémentation supplémentaire : TaskStatsView

Au-delà des vues d'ajout et de liste, une `TaskStatsView` permettrait d'offrir une vue d'ensemble précieuse sur l'état des tâches, offrant des métriques clés pour un suivi efficace.

Affichage des statistiques clés

La méthode `render(stats)` affichera dynamiquement les informations suivantes :

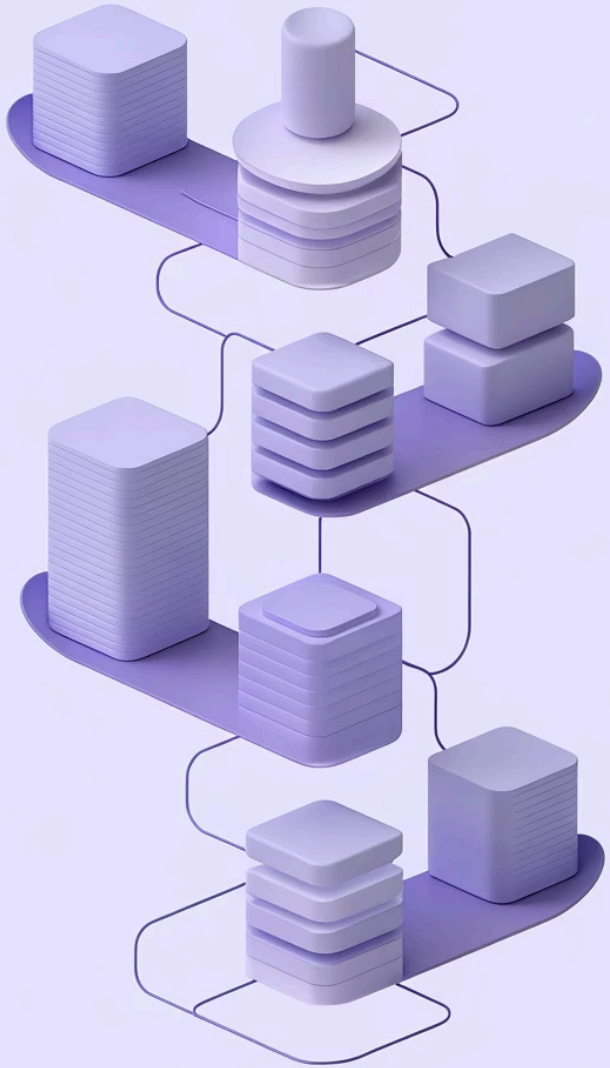
- Le nombre total de tâches.
- Le nombre de tâches complétées.
- Le nombre de tâches actives.
- Le taux de complétion sous forme de pourcentage.

Présentation visuelle

Les statistiques seront présentées dans une grille de cartes pour une lisibilité accrue. Le taux de complétion sera également visualisé par une barre de progression intuitive.

Contrôle de la visibilité

La vue exposera des méthodes `hide()` et `show()` pour permettre au contrôleur de gérer facilement l'affichage et le masquage de ces statistiques selon le besoin.



MODULE

Le contrôleur (controller) - Coordination et orchestration

MODULE

Le contrôleur : Explication synthétique

Le contrôleur agit comme le chef d'orchestre central de l'application, assurant la cohésion entre les différentes parties et répondant aux interactions.

Coordination

Il orchestre l'interaction fluide entre le modèle (données) et la vue (interface utilisateur).

Écoute des événements de la vue

Il capte les actions de l'utilisateur (clics, saisies) pour les interpréter et y réagir.

Surveillance du modèle

Il est attentif aux changements de données dans le modèle pour savoir quand l'interface doit être mise à jour.

Interaction avec le modèle

Il invoque les méthodes du modèle pour effectuer des opérations sur les données (ajout, modification, suppression).

Mise à jour de la vue

Après un changement de données, il demande à la vue de se rafraîchir pour refléter l'état actuel.

Gestion de la persistance

Il s'occupe de sauvegarder l'état de l'application, par exemple, dans le `localStorage` du navigateur.

Exploration du stockage côté client : LocalStorage et SessionStorage

Pour la persistance des données directement dans le navigateur, JavaScript offre des mécanismes intégrés. Comprendre leurs différences est essentiel pour choisir la bonne solution de stockage.

LocalStorage : Persistance des données

`localStorage` permet de stocker des données de manière permanente dans le navigateur. Ces données ne sont pas supprimées lorsque le navigateur est fermé et sont accessibles même après un redémarrage.

- **Durée de vie :** Persiste jusqu'à ce qu'elles soient explicitement supprimées par l'utilisateur ou le code.
- **Portée :** Disponible pour toutes les pages du même domaine, protocole et port.
- **Cas d'usage :** Préférences utilisateur, thèmes, données de panier d'achat, données d'application hors ligne.

SessionStorage : Persistance temporaire

`sessionStorage` est similaire à `localStorage`, mais les données stockées sont temporaires. Elles sont effacées dès que l'utilisateur ferme l'onglet ou la fenêtre du navigateur où les données ont été définies.

- **Durée de vie :** Valide uniquement pour la durée de la session de la page (jusqu'à la fermeture de l'onglet).
- **Portée :** Spécifique à l'onglet ou à la fenêtre qui a créé les données (même pour le même domaine).
- **Cas d'usage :** Saisie de formulaires multi-étapes, état d'une session de navigation, informations temporaires d'une transaction.

MODULE

Les règles d'or du contrôleur

Pour garantir un découplage efficace et une architecture MVC robuste, le contrôleur doit adhérer à des principes clairs, en se concentrant sur la coordination et en évitant les responsabilités du modèle ou de la vue.

✅ Connaît le modèle et la vue

Il agit comme un pont, ayant la connaissance des deux pour orchestrer leurs interactions.

✅ Coordonne les interactions

Sa fonction principale est de diriger le flux de données et d'actions entre le modèle et la vue.

✅ Gère le flux de données

Il est responsable de récupérer les données du modèle et de les transmettre à la vue pour l'affichage, et vice-versa.

❌ Ne manipule pas directement le DOM

C'est le rôle exclusif de la vue de gérer l'interface utilisateur et ses éléments.

❌ Ne contient pas de logique métier

La logique métier appartient au modèle, qui doit être indépendant de la présentation et de l'orchestration.

ANALOGIE

Le contrôleur : Le chef d'orchestre

Le contrôleur est comme un chef d'orchestre : il ne joue aucun instrument, mais il dirige l'ensemble pour créer une harmonie parfaite.

Un simple coordinateur

Il ne manipule pas directement les éléments techniques (pas de manipulation du DOM) et ne contient pas de logique métier complexe.

Harmonise model et vue

Il coordonne les "musiciens" (le modèle et la vue) pour que leurs actions soient synchronisées.

Écoute attentive

Il capte les "notes" (événements utilisateur) de la vue et les interprète pour savoir quoi faire.

Donne des directives

Il envoie des "instructions" au modèle pour modifier les données ou à la vue pour rafraîchir l'affichage.

Orchestration d'événements

Quand une "vue" signale une interaction, le contrôleur s'assure que le "modèle" réponde de manière appropriée.

MODULE

Exemple de code : Le contrôleur

Le contrôleur `ProductController` connecte la logique métier du `ProductManager` (modèle) à l'interface utilisateur gérée par `ProductListView` (vue), tout en orchestrant les interactions utilisateur.

▼ **Classe ProductController (contrôleur)**

```
class ProductController {
  #model;
  #view;

  constructor() {
    this.#model = new ProductManager();
    this.#view = new ProductListView("#product-list");

    this.#init();
  }

  #init() {
    // Écoute les événements du modèle (changement de données)
    this.#model.on("product:added", () => {
      this.#updateView();
    });

    this.#model.on("product:removed", () => {
      this.#updateView();
    });

    // Écoute les événements de la vue (actions utilisateur)
    this.#view.onDelete((productId) => {
      this.#handleDelete(productId);
    });

    // Chargement initial de la vue avec les données du modèle
    this.#updateView();
  }

  // Gestionnaire d'événement pour la suppression d'un produit
  #handleDelete(productId) {
    if (confirm("Supprimer ce produit ?")) {
      this.#model.remove(productId);
      // Pas besoin d'appeler #updateView() ici,
      // car le contrôleur écoute déjà "product:removed" du modèle.
    }
  }

  // Méthode pour demander à la vue de se rafraîchir
  #updateView() {
    const products = this.#model.getAll();
    this.#view.render(products);
  }
}
```

MISE EN PRATIQUE

Création du TaskController

L'objectif est de développer le `TaskController`, le cœur de notre application MVC, en suivant des spécifications précises pour orchestrer les interactions entre le modèle et les vues.

1

Initialisation

Le contrôleur doit **instancier** le `TaskManager` (modèle) et toutes les vues (`TaskFormView`, `TaskListView`) nécessaires à l'application.

2

Gestion des Événements

Il est responsable de la **configuration des callbacks** pour écouter les événements du `TaskManager` (changements de données) et des vues (actions utilisateur comme l'ajout, la modification ou la suppression de tâches).

3

Synchronisation de la Vue

Le contrôleur doit implémenter une méthode `#updateAllViews()` pour **rafraîchir toutes les vues** chaque fois que les données du modèle sont modifiées, assurant la cohérence de l'interface utilisateur.

4

Persistance des Données

Il gère les fonctions de **sauvegarde** (`#saveToStorage()`) et de **chargement** (`#loadFromStorage()`) des données, y compris le chargement initial au démarrage de l'application.

MISE EN PRATIQUE

Étapes de développement du TaskController

Créer le fichier

Définir la structure du fichier `src/controllers/TaskController.js`.

Définir les propriétés privées

Encapsuler `#manager` et `#views` pour un meilleur contrôle d'accès.

Instancier Manager et Vues

Initialiser le `TaskManager` et les objets des vues dans le constructeur.

Méthode `#init()`

Créer une méthode d'initialisation pour regrouper la configuration.

Callbacks du manager

Écouter les événements du `TaskManager` (ex : "tasks:changed") pour réagir aux modifications des données.

Callbacks des vues

Capturer les événements déclenchés par l'utilisateur via les vues (clics, soumissions).

Gestionnaires d'événements

Implémenter les fonctions qui réagiront aux événements du manager et des vues.

Implémenter `#updateAllViews()`

Créer une méthode pour rafraîchir toutes les vues de l'application.

MISE EN PRATIQUE

À vous de continuer le TaskController

Implémenter la persistance

Mettre en œuvre les fonctions `#saveToStorage()` et `#loadFromStorage()`.

Chargement initial

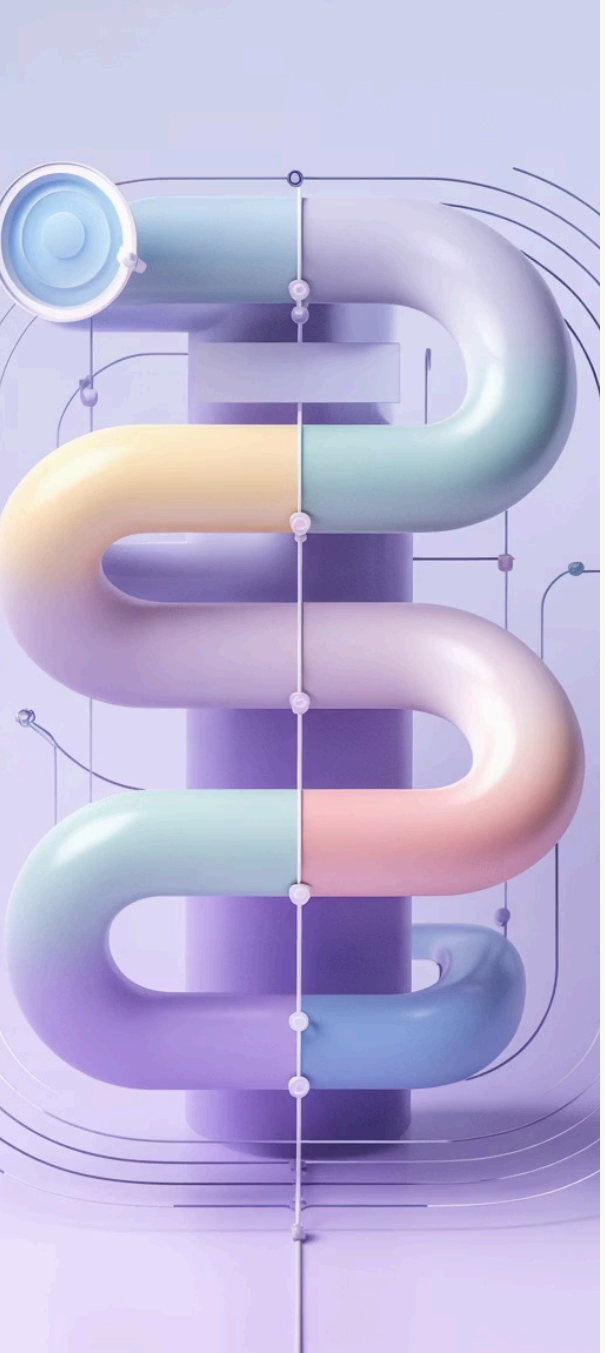
Appeler `#loadFromStorage()` au démarrage de l'application pour restaurer l'état.

CONCLUSION

Et… on s'arrête là pour les notions du jour !

Ça fait beaucoup (vraiment), donc on fait une pause pour aujourd'hui.

Et cette fois-ci, on s'arrête pour de vrai ! Pas de projet de fin de journée.



À SUIVRE

Prochaines étapes

L'essentiel du programme JavaScript est désormais terminé ! *(gros morceau !)*

Mais il reste encore quelques petites choses à découvrir, notamment l'API Canvas et TypeScript.

Rassurez-vous : l'objectif est **uniquement de découvrir** ces outils et technologies, pas de les maîtriser 😉

> **Spoiler :** TypeScript, c'est du JavaScript moins "pâte-à-modeler teuteu".
>
> *Gauthier, janvier 2026*